What drives the price of a car?

OVERVIEW

In this application, you will explore a dataset from kaggle. The original dataset contained information on 3 million used cars. The provided dataset contains information on 426K cars to ensure speed of processing. Your goal is to understand what factors make a car more or less expensive. As a result of your analysis, you should provide clear recommendations to your client -- a used car dealership -- as to what consumers value in a used car.

CRISP-DM Framework

To frame the task, throughout our practical applications we will refer back to a standard process in industry for data projects called CRISP-DM. This process provides a framework for working through a data problem. Your first step in this application will be to read through a brief overview of CRISP-DM here. After reading the overview, answer the questions below.

Business Understanding

From a business perspective, we are tasked with identifying key drivers for used car prices. In the CRISP-DM overview, we are asked to convert this business framing to a data problem definition. Using a few sentences, reframe the task as a data task with the appropriate technical vocabulary.

In [ ]:

```
```

The business task of identifying key drivers for used car prices can be reframed as a data task of performing a regression analysis on a dataset of used car transactions to determine the variables that have the most significant impact on the sale price of a used car. This involves collecting and cleaning data on various features of the car, such as make, model, year, mileage, condition, and location, as well as external factors that may influence the price, such as economic indicators, consumer demand, and market trends. The data task also involves selecting an appropriate statistical model and methodology, and evaluating the models accuracy and predictive power using various metrics such as R-squared, mean absolute error, and root mean square error.

In [ ]:

```
```

Data Understanding

After considering the business understanding, we want to get familiar with our data. Write down some steps that you would take to get to know the dataset and identify any quality issues within. Take time to get to know the dataset and explore what information it contains and how this could be used to inform your business understanding.

In [ ]:

```
```

Data exploration

To get to know the dataset and identify any quality issues within, I would take the following steps:

1/ Obtain the dataset and read the data documentation: Get access to the dataset and read any available documentation that describes the data's structure, source, and format. This step will help to understand the data's background, and what each column represents.

2/ Explore the data: Examine the dataset by computing descriptive statistics (mean, median, mode, standard deviation, minimum, maximum) for each column. This will provide a general idea of the data range and any potential issues, such as outliers or missing values.

3/ Visualize the data: Generate different types of graphs, charts, and plots to better understand the dataset's distribution, patterns, and relationships between different variables. This will help to detect any anomalies or trends within the data.

4/ Check for missing data: Identify any missing values in the dataset, assess the percentage of missing data, and decide on the appropriate imputation strategy.

5/ Check for duplicates: Look for any duplicates within the dataset and determine the appropriate action to handle them.

6/ Identify outliers: Check for outliers within the dataset and determine if they are erroneous or genuine values. Decide on the appropriate action to handle outliers.

7/ Verify data accuracy: Verify that the data accurately reflects the real-world phenomenon and ensure that the data is consistent with the business requirements.

By conducting these steps, it will be possible to identify any quality issues within the dataset and clean the data, ensuring it is ready for use in subsequent data analysis and modeling.

In [ ]:

```
```

In [1]:

```
# #  Steps followed in this analysis
# 1. Import the required libraries
# 2. Import the data
```

```
# 3. Data Preparation
# 4. Feature Engineering - Handle outliers, handle missing data, handle Categor
# 5. Visualize the relationship
# 6. Prepare for model training
# 7. Model building and eavaluation
# 7. Check for accuracy - MAE, MSE, RMSE, R2
```

In [2]:
```
# import the required libraries for the analysis
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from sklearn.model_selection import train_test_split
import warnings
import seaborn as sns
import plotly.express as px
from sklearn.preprocessing import PolynomialFeatures, StandardScaler, OneHotEnc


warnings.filterwarnings('ignore')
```

In [3]:
```
#  Import the data and review it for the initiatial preparation for analysis
usedcar =  pd.read_csv('/Users/mohul1/Downloads/practical_application_II_starte
```

In [4]:
```
usedcar.head()
```

Out[4]:

	id	region	price	year	manufacturer	model	condition	cylinders	fuel	odom
0	7222695916	prescott	6000	NaN	NaN	NaN	NaN	NaN	NaN	
1	7218891961	fayetteville	11900	NaN	NaN	NaN	NaN	NaN	NaN	
2	7221797935	florida keys	21000	NaN	NaN	NaN	NaN	NaN	NaN	
3	7222270760	worcester / central MA	1500	NaN	NaN	NaN	NaN	NaN	NaN	
4	7210384030	greensboro	4900	NaN	NaN	NaN	NaN	NaN	NaN	

In [5]:
```
usedcar.tail()
```

Out[5]:

	id	region	price	year	manufacturer	model	condition	cylinders
426875	7301591192	wyoming	23590	2019.0	nissan	maxima s sedan 4d	good	6 cylinders
426876	7301591187	wyoming	30590	2020.0	volvo	s60 t5 momentum sedan 4d	good	NaN
426877	7301591147	wyoming	34990	2020.0	cadillac	xt4 sport suv 4d	good	NaN
426878	7301591140	wyoming	28990	2018.0	lexus	es 350 sedan 4d	good	6 cylinders
426879	7301591129	wyoming	30590	2019.0	bmw	4 series 430i gran coupe	good	NaN

In [6]:

```
# data types and number of rows & columns
usedcar.info(), usedcar.shape
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 426880 entries, 0 to 426879
Data columns (total 18 columns):
 #   Column        Non-Null Count   Dtype  
---  ------        --------------   -----  
 0   id            426880 non-null  int64  
 1   region        426880 non-null  object 
 2   price         426880 non-null  int64  
 3   year          425675 non-null  float64
 4   manufacturer  409234 non-null  object 
 5   model         421603 non-null  object 
 6   condition     252776 non-null  object 
 7   cylinders     249202 non-null  object 
 8   fuel          423867 non-null  object 
 9   odometer      422480 non-null  float64
 10  title_status  418638 non-null  object 
 11  transmission  424324 non-null  object 
 12  VIN           265838 non-null  object 
 13  drive         296313 non-null  object 
 14  size          120519 non-null  object 
 15  type          334022 non-null  object 
 16  paint_color   296677 non-null  object 
 17  state         426880 non-null  object 
dtypes: float64(2), int64(2), object(14)
memory usage: 58.6+ MB
```

Out[6]:

```
(None, (426880, 18))
```

In [7]:

```
# #  Quick glance on the data:
# There are 426880 entries for 18 features of used cars
# There are 4 columns of integer/float datatype whereas rest 14 are object data

# # Next steps:-
# 1. Find and remove null values
# 2. Remove unwanted columns/features from the dataset
# 3. Varify categorical features
```

In [8]:

```
#  Find out the total null values
usedcar.isnull().sum()
```

Out[8]:
```
id                  0
region              0
price               0
year             1205
manufacturer    17646
model            5277
condition      174104
cylinders      177678
fuel             3013
odometer         4400
title_status     8242
transmission     2556
VIN            161042
drive          130567
size           306361
type            92858
paint_color    130203
state               0
dtype: int64
```

In [ ]:

In [9]:
```
usedcar.describe()

# PRICE AND ODOMETER SEEM TO HAVE SOME OUTLIERS AS THE MAX VALUE IS EXCEPTIONAI
```

Out[9]:

	id	price	year	odometer
count	4.268800e+05	4.268800e+05	425675.000000	4.224800e+05
mean	7.311487e+09	7.519903e+04	2011.235191	9.804333e+04
std	4.473170e+06	1.218228e+07	9.452120	2.138815e+05
min	7.207408e+09	0.000000e+00	1900.000000	0.000000e+00
25%	7.308143e+09	5.900000e+03	2008.000000	3.770400e+04
50%	7.312621e+09	1.395000e+04	2013.000000	8.554800e+04
75%	7.315254e+09	2.648575e+04	2017.000000	1.335425e+05
max	7.317101e+09	3.736929e+09	2022.000000	1.000000e+07

Data Preparation

After our initial exploration and fine tuning of the business understanding, it is time to construct our final dataset prior to modeling. Here, we want to make sure to handle any integrity issues and cleaning, the engineering of new features, any transformations that we believe should happen (scaling, logarithms, normalization, etc.), and general preparation for modeling with `sklearn`.

In [10]:
```
# price - there are quite a bit of outliers
print(f'Number of rows without any price: ',(usedcar[usedcar['price'] == 0]).sh
print(f'Number of rows price less than $100: ',(usedcar[usedcar['price'] < 100]

# drop 3090 rows with the price less than 100
```

```
usedcar.drop(usedcar[usedcar['price']<100].index, inplace=True)
usedcar.drop(usedcar[usedcar['price']>=999999].index, inplace=True)

usedcar.shape
```

```
Number of rows without any price:  32895
Number of rows price less than $100:  36222
```

Out[10]:
```
(390597, 18)
```

In [11]:
```
#further outliers that do no make sense- cars with such high odometer reading a
# this is more than car's life
drop_odometer_index = usedcar[usedcar.odometer >= 1000000].index
print(drop_odometer_index)

# DROP THESE VALUES

usedcar.drop((drop_odometer_index),inplace=True )
usedcar.shape
```

```
Int64Index([   455,    562,    616,   1115,   1440,   1729,   4614,   7657,
              9218,   9878,
            ...
            411561, 412634, 413267, 417631, 417643, 419355, 419494, 422547,
            423091, 426106],
           dtype='int64', length=502)
```

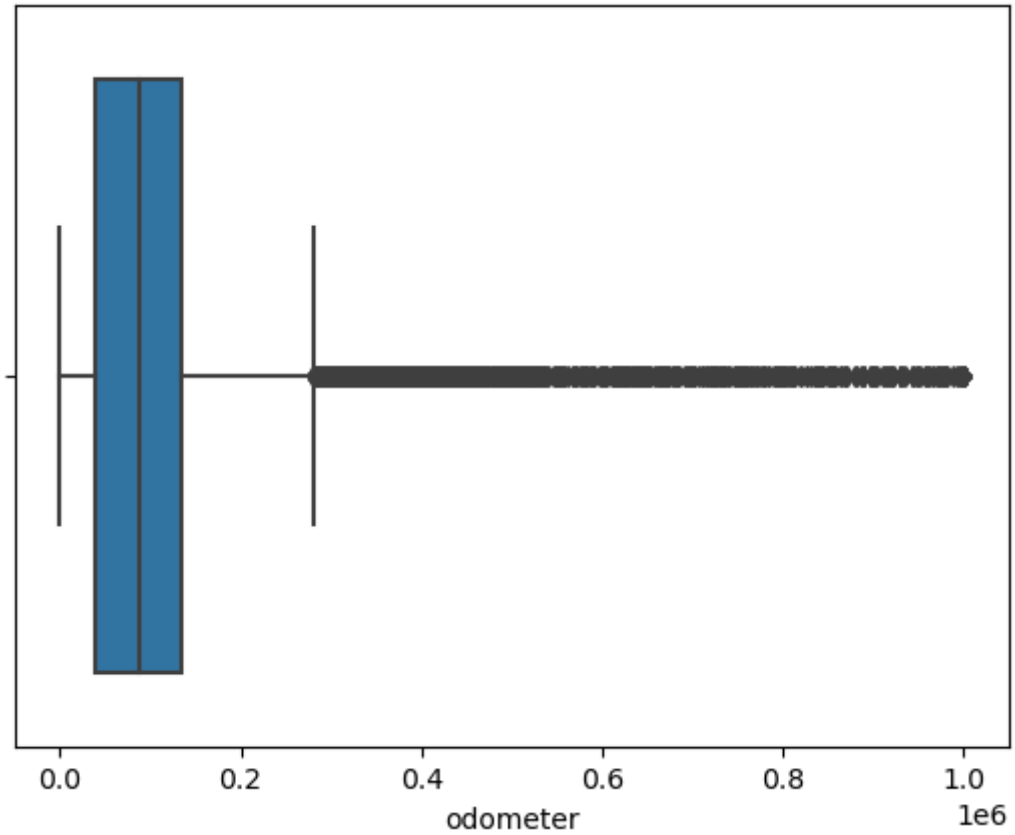
Out[11]:
```
(390095, 18)
```

In [12]:
```
usedcar.info()
```

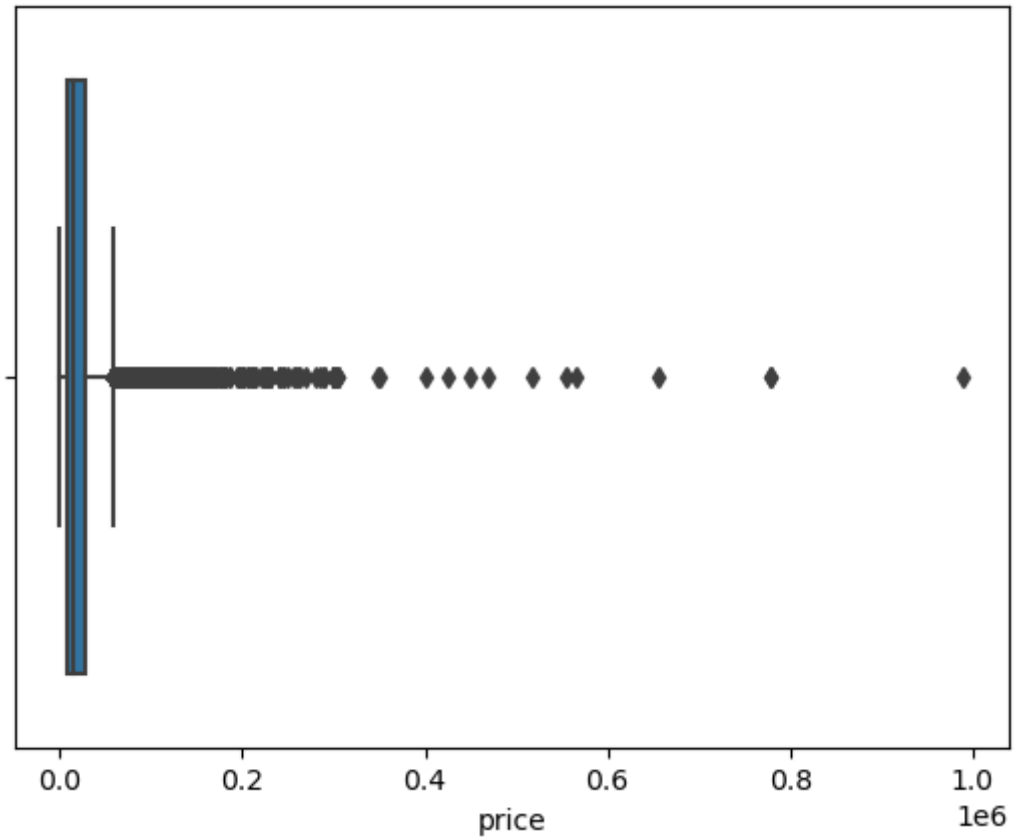
```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 390095 entries, 0 to 426879
Data columns (total 18 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   id            390095 non-null  int64
 1   region        390095 non-null  object
 2   price         390095 non-null  int64
 3   year          388922 non-null  float64
 4   manufacturer  374391 non-null  object
 5   model         385553 non-null  object
 6   condition     240647 non-null  object
 7   cylinders     231713 non-null  object
 8   fuel          387502 non-null  object
 9   odometer      387813 non-null  float64
 10  title_status  382368 non-null  object
 11  transmission  388274 non-null  object
 12  VIN           239984 non-null  object
 13  drive         271351 non-null  object
 14  size          108884 non-null  object
 15  type          306023 non-null  object
 16  paint_color   274610 non-null  object
 17  state         390095 non-null  object
dtypes: float64(2), int64(2), object(14)
memory usage: 56.5+ MB
```

In [13]:
```
# # # Quick glance of the data with plot for all features
# sns.pairplot(usedcar)
# plt.show()
```

In [14]:
```
# for outliers in int fields - ODOMETER
sns.boxplot(usedcar['odometer'])
plt.show()
```



In [15]:
```
# for outliers in int fields - PRICE
sns.boxplot(usedcar['price'])
plt.show()
```

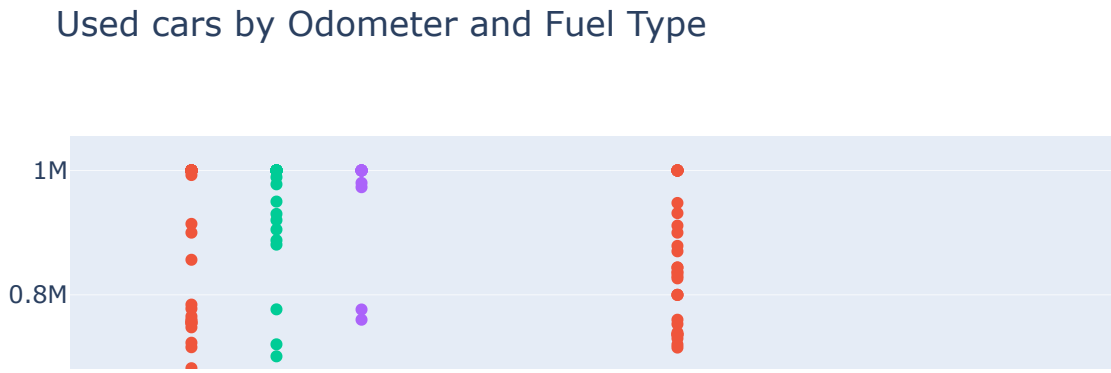


In [16]:
```
#  usedcar[usedcar.price >= 999999]
```

In [17]:
```
usedcar['fuel'].unique()
```

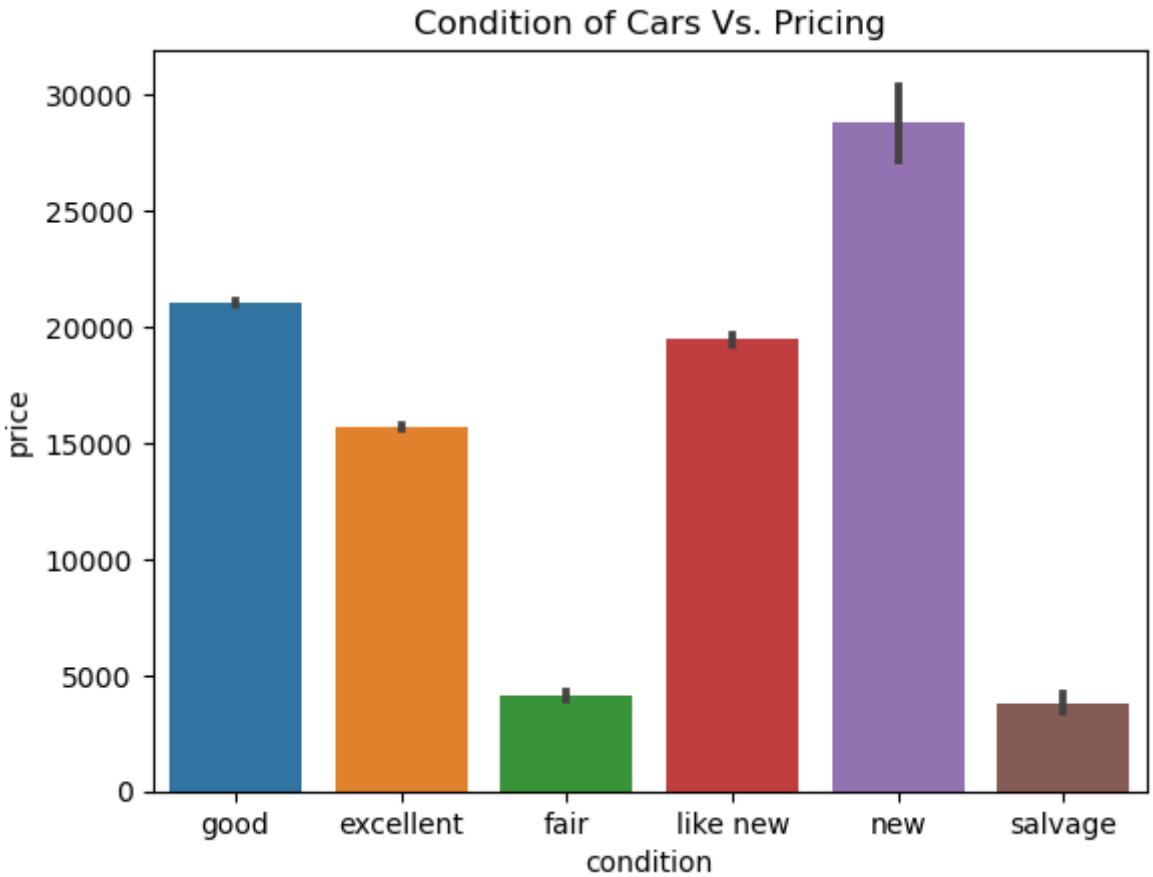
Out[17]:
```
array([nan, 'gas', 'other', 'diesel', 'hybrid', 'electric'], dtype=object)
```

In [18]:
```
# A Quick View into usedcar by Odometer vs. Fuel Type. This may indicate which
#box plot by odometer and fuel type
import plotly.express as px
fig = px.box(usedcar, x="fuel", y="odometer", color="size", title='Used cars by

# fig = px.scatter(usedcar, x='fuel', y="odometer")
fig.show()
```

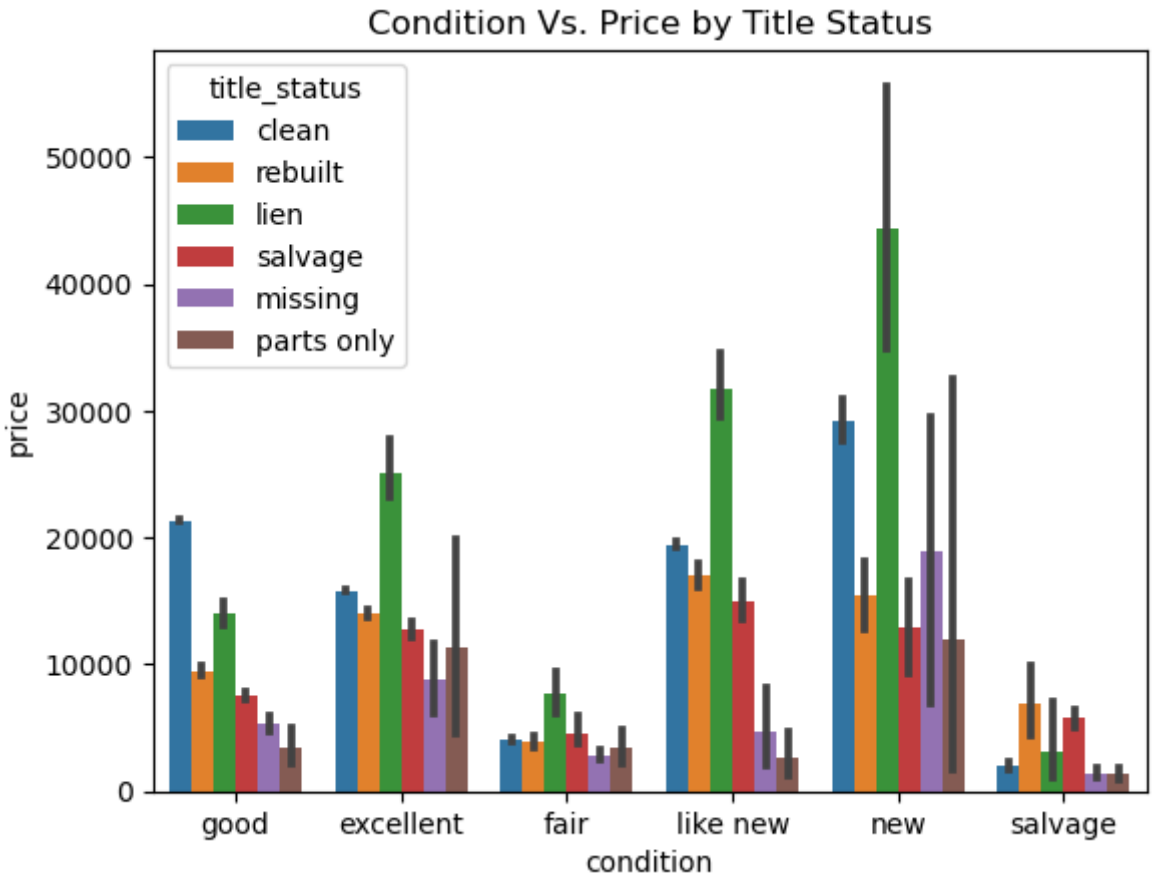


In [19]:
```
# checking with "year" and "manfacturer"
fig = px.histogram(usedcar, x="year" , color='manufacturer', nbins=20, title="C
fig.show()

# this shows "FORD" has highest market share followed by "CHEVROLET"
```

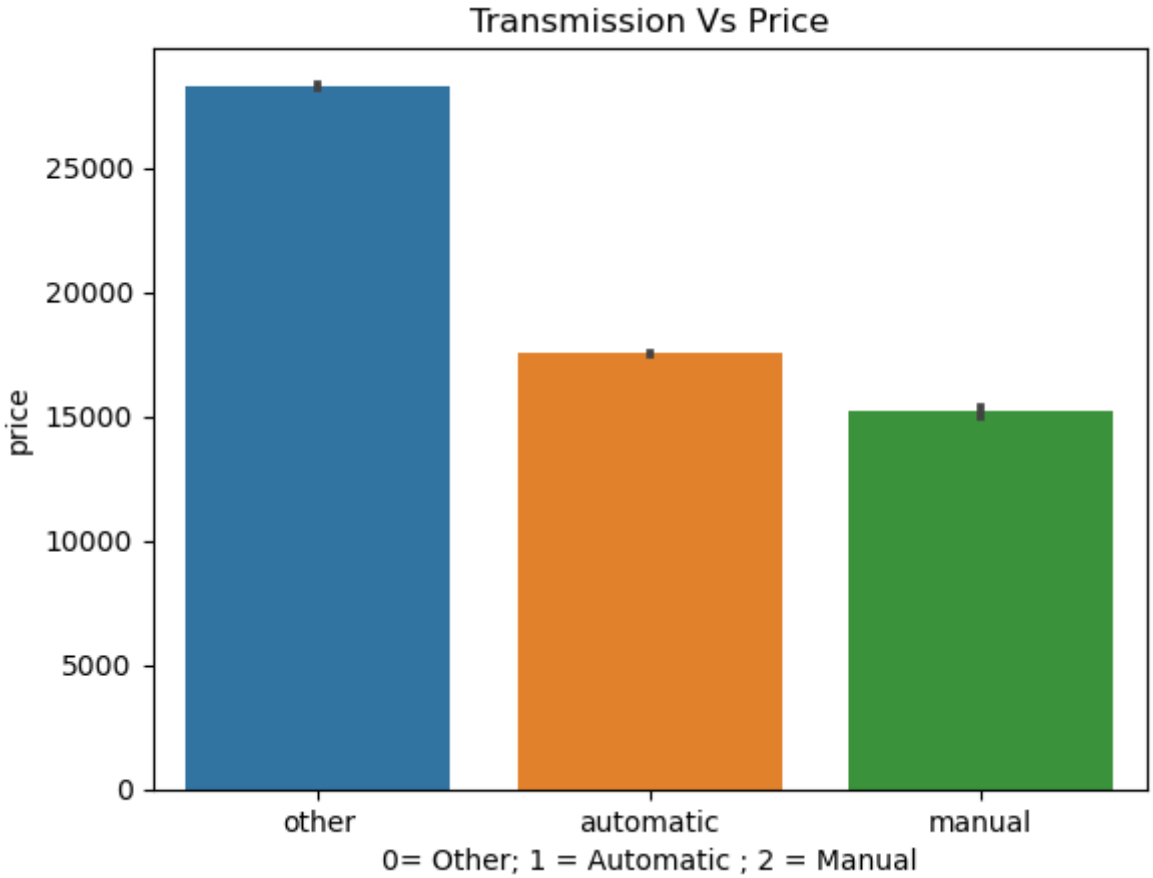
In [20]:
```
# Condition of used car and its influence on price
sns.barplot(data=usedcar, x='condition', y='price')
plt.title('Condition of Cars Vs. Pricing')
plt.show()

# sns.boxplot(data=data, x='condition', y='price').set(title='Condition vs Pric

# NEW/ LIKE NEW / GOOD conditined cars in demand compared to other used cars
```



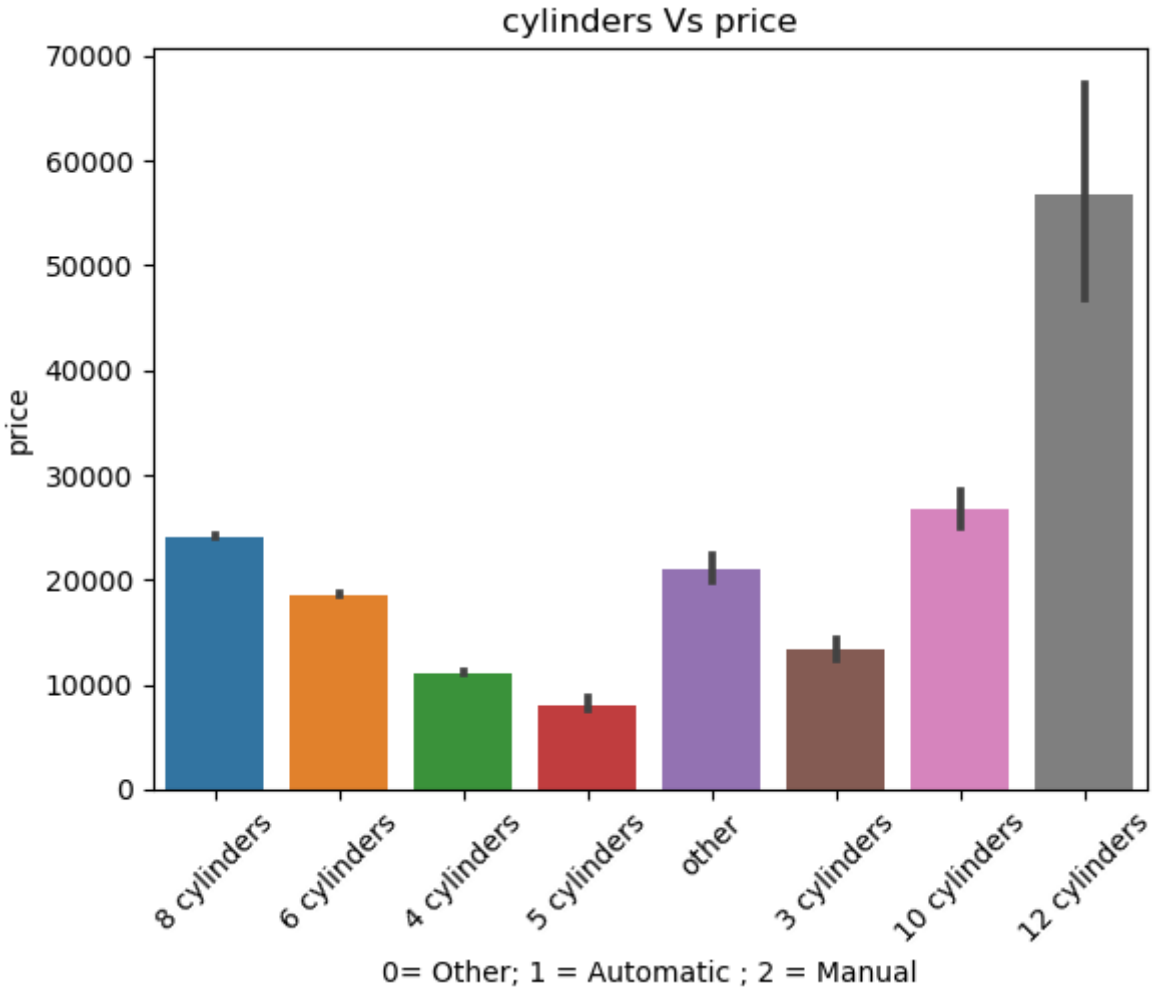
In [21]:
```
# A Look Into The Condition vs Title vs Price
sns.barplot(data=usedcar, x='condition', y='price', hue='title_status')
plt.title('Condition Vs. Price by Title Status')
plt.show()

# "LIEN" is leased car which is higher in most of the used car condition catego
```



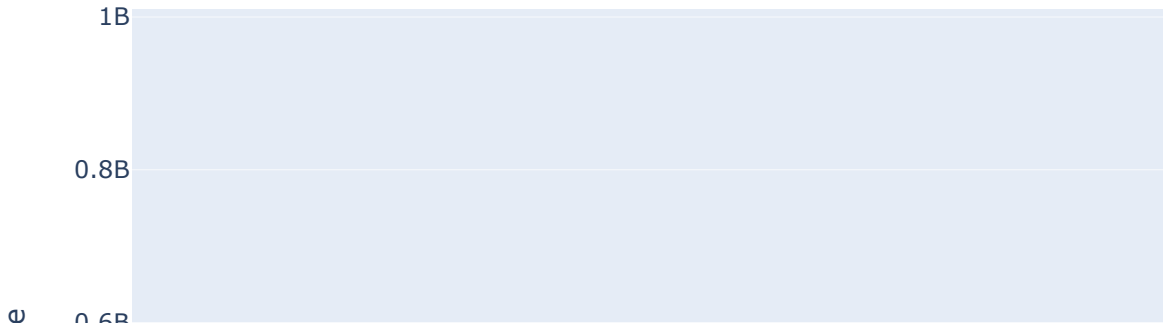
In [22]:
```
#Transmission of used car and its influence on price
sns.barplot(data=usedcar, x='transmission', y='price')
plt.title('Transmission Vs Price')
plt.xlabel("0= Other; 1 = Automatic ; 2 = Manual")
plt.show()
```



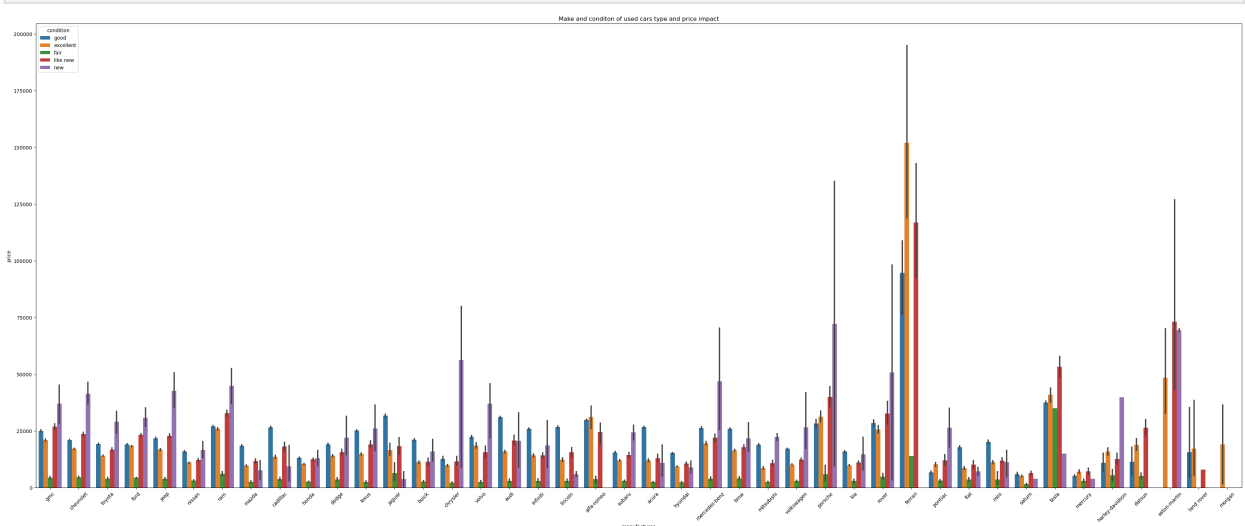
In [23]:
```
#Transmission of used car and its influence on price
sns.barplot(data=usedcar, x='cylinders', y='price')
plt.title('cylinders Vs price')
plt.xlabel("0= Other; 1 = Automatic ; 2 = Manual")
plt.xticks(rotation=45)
plt.show()
```



In [24]:
```
# Year Vs. Price Distribution
# ##sns does not give the clear plot hence using plotly
# sns.barplot(data=usedcar, x='year', y='price')
# plt.title('year Vs price')

px.histogram(data_frame=usedcar, x='year', y='price')
```



In [25]:
```
#Manufacturer with car type and its impact to price
from matplotlib import rcParams
rcParams['figure.figsize'] = 44,17
sns.barplot(data=usedcar.query("condition in ['excellent', 'good', 'like new',
plt.xticks(rotation=45)
plt.title('Make and conditon of used cars type and price impact')
plt.show()
```



In [26]:
```
usedcar.shape, usedcar.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 390095 entries, 0 to 426879
Data columns (total 18 columns):
 #   Column        Non-Null Count   Dtype  
---  ------        --------------   -----  
 0   id            390095 non-null  int64  
 1   region        390095 non-null  object 
 2   price         390095 non-null  int64  
 3   year          388922 non-null  float64
 4   manufacturer  374391 non-null  object 
 5   model         385553 non-null  object 
 6   condition     240647 non-null  object 
 7   cylinders     231713 non-null  object 
 8   fuel          387502 non-null  object 
 9   odometer      387813 non-null  float64
 10  title_status  382368 non-null  object 
 11  transmission  388274 non-null  object 
 12  VIN           239984 non-null  object 
 13  drive         271351 non-null  object 
 14  size          108884 non-null  object 
 15  type          306023 non-null  object 
 16  paint_color   274610 non-null  object 
 17  state         390095 non-null  object 
dtypes: float64(2), int64(2), object(14)
memory usage: 56.5+ MB
```

Out[26]:

```
((390095, 18), None)
```

In [27]:

```
# Check on the unique counts of the object columns

unique_counts_df = pd.DataFrame(columns=['feature', 'unique_count'])

for column in usedcar.select_dtypes(include='object').columns:
    unique_counts_df.loc[len(unique_counts_df.index)] = [column, usedcar[colum

unique_counts_df
```

Out[27]:

	feature	unique_count
0	region	404
1	manufacturer	42
2	model	27975
3	condition	6
4	cylinders	8
5	fuel	5
6	title_status	6
7	transmission	3
8	VIN	105438
9	drive	3
10	size	4
11	type	13
12	paint_color	12
13	state	51

In [28]:

```
# Here replacing categorical features, which has less than 5 values, with numer
# 5 categorical values, I am going to use "one hot encoder" method


#Assigning numeric value for "drive" categorical feature
usedcar['drive'] = usedcar['drive'].str.replace(r'4wd','1')
usedcar['drive'] = usedcar['drive'].str.replace(r'fwd','1')
usedcar['drive'] = usedcar['drive'].str.replace(r'rwd','2')
usedcar[['drive']] = usedcar[['drive']].fillna('0')
usedcar.drive = usedcar.drive.astype(np.int64)




#similary replacing "transmission" with numeric values
usedcar['transmission'] = usedcar['transmission'].str.replace(r'other','0')
usedcar['transmission'] = usedcar['transmission'].str.replace(r'automatic','1')
usedcar['transmission'] = usedcar['transmission'].str.replace(r'manual','2')
usedcar[['transmission']] = usedcar[['transmission']].fillna('0')
usedcar.transmission = usedcar.transmission.astype(np.int64)



#Assigning numeric value for "fuel" categorical feature
usedcar['fuel'] = usedcar['fuel'].str.replace(r'gas','1')
usedcar['fuel'] = usedcar['fuel'].str.replace(r'hybrid','1')
usedcar['fuel'] = usedcar['fuel'].str.replace(r'diesel','2')
usedcar['fuel'] = usedcar['fuel'].str.replace(r'other','3')
usedcar['fuel'] = usedcar['fuel'].str.replace(r'electric','4')
usedcar[['fuel']] = usedcar[['fuel']].fillna('0')
usedcar.drive = usedcar.drive.astype(np.int64)
```

```
#Assigning numeric value for "size" categorical feature
usedcar['size'] = usedcar['size'].str.replace(r'compact','1')
usedcar['size'] = usedcar['size'].str.replace(r'mid-size','1')
usedcar['size'] = usedcar['size'].str.replace(r'full-size','2')
usedcar['size'] = usedcar['size'].str.replace(r'sub-compact','3')
usedcar[['size']] = usedcar[['fuel']].fillna('0')
usedcar.drive = usedcar.drive.astype(np.int64)
```

In [29]:

```
# BASED ON ABOVE INFORMATION, THERE ARE MULTIPLE FEATRES WITH NULL VALUES FOR
# , 'CYLINDERS', 'FUEL', 'ODOMETER', 'TITLE_STATUS', 'TRANSMISSION', 'VIN', 'DI
# IS NO RELIABLE WAY TO REPLACE THESE NULL DATA WITHOUT INTRODUCING SOME BIAS T
# AFTER CAREFULLY ANALYSIS OF MISSING DATA, WE NEED TO MORPH THE DATA FOR THOSE
# PREDICTION AND DROP FOR OTHERS.


# VIN identifies the unique car and has a lot of null values. It does not impac
# the can be safely removed
# df = usedcar['VIN'].dropna()

# MODEL feature has 29k+ unique values which can not be possibly categorized. A
# prediction but still better to drop to avoid introducing error/bias to the mo

df = usedcar.dropna(subset = ['VIN', 'model'])
```

In [30]:

```
# Check on the unique counts after removing null values from 'VIN' and 'model'
unique_counts_clean = pd.DataFrame(columns=['feature', 'unique_count'])

for column in df.select_dtypes(include='object').columns:
    unique_counts_clean.loc[len(unique_counts_clean.index)] = [column, df[colum

unique_counts_clean
```

Out[30]:

	feature	unique_count
0	region	404
1	manufacturer	41
2	model	16905
3	condition	6
4	cylinders	8
5	fuel	5
6	title_status	6
7	VIN	104945
8	size	5
9	type	13
10	paint_color	12
11	state	51

In [31]:

```
# key driving factor for used car price should be:-

# Based on the above analysis and availability of data, we choose the following
```

```
# 'price', 'year', 'manufacturer', 'model', 'condition', 'cylinders', 'fuel',
# 'transmission',  'type', 'drive'as input features and price as the obvious ta

#Extracting features that will be used for model development into a separate da
df = usedcar[['price', 'year', 'manufacturer', 'model', 'condition', 'cylinders

#dropping NaN values from the dataframe df
df=df.dropna()
df.shape
```

Out[31]:

```
(140442, 12)
```

In [32]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 140442 entries, 27 to 426878
Data columns (total 12 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   price         140442 non-null  int64
 1   year          140442 non-null  float64
 2   manufacturer  140442 non-null  object
 3   model         140442 non-null  object
 4   condition     140442 non-null  object
 5   cylinders     140442 non-null  object
 6   fuel          140442 non-null  object
 7   odometer      140442 non-null  float64
 8   title_status  140442 non-null  object
 9   transmission  140442 non-null  int64
 10  type          140442 non-null  object
 11  drive         140442 non-null  int64
dtypes: float64(2), int64(3), object(7)
memory usage: 13.9+ MB
```

In [ ]:

In [33]:

```
df.describe()
```

Out[33]:

	price	year	odometer	transmission	drive
count	140442.000000	140442.000000	140442.000000	140442.000000	140442.000000
mean	17285.598112	2010.032505	101128.945380	0.900016	1.062666
std	13804.176696	9.364510	68140.506473	0.468776	0.578138
min	100.000000	1900.000000	0.000000	0.000000	0.000000
25%	6500.000000	2007.000000	45958.000000	1.000000	1.000000
50%	12995.000000	2012.000000	97113.000000	1.000000	1.000000
75%	25995.000000	2016.000000	143656.500000	1.000000	1.000000
max	349999.000000	2022.000000	999999.000000	2.000000	2.000000

In [34]:

```
df.head()
```

Out[34]:

	price	year	manufacturer	model	condition	cylinders	fuel	odometer	title_status	t
27	33590	2014.0	gmc	sierra 1500 crew cab slt	good	8 cylinders	1	57923.0	clean	
28	22590	2010.0	chevrolet	silverado 1500	good	8 cylinders	1	71229.0	clean	
29	39590	2020.0	chevrolet	silverado 1500 crew	good	8 cylinders	1	19160.0	clean	
30	30990	2017.0	toyota	tundra double cab sr	good	8 cylinders	1	41124.0	clean	
31	15000	2013.0	ford	f-150 xlt	excellent	6 cylinders	1	128000.0	clean	

In [35]:

```
# categorical values in rest of the object columns
object_col = ['cylinders', 'fuel',
       'title_status', 'transmission', 'drive',  'type']

for i in object_col:
    print('=>',i,'<=')
    print(df[i].unique().tolist())
    print(' ')
```

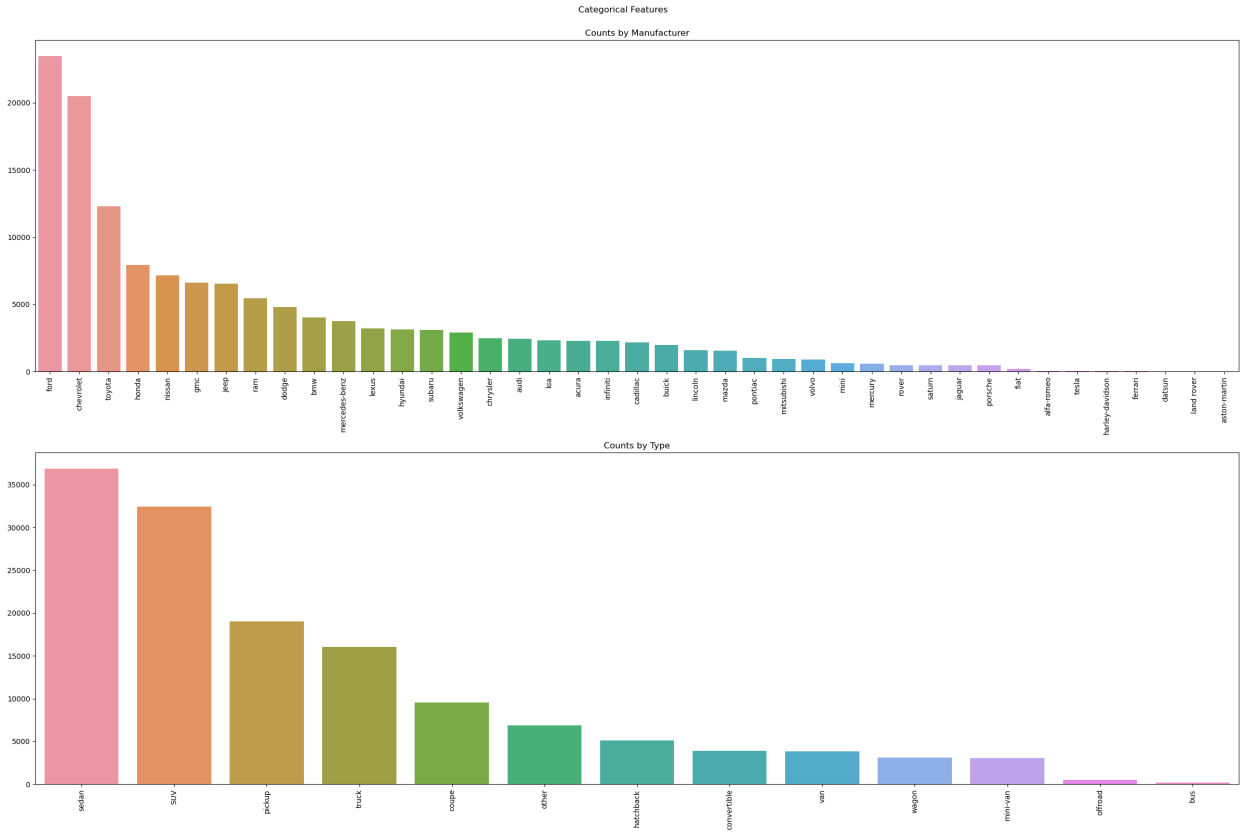
```
=> cylinders <=
['8 cylinders', '6 cylinders', '4 cylinders', '5 cylinders', '10 cylinders',
'3 cylinders', 'other', '12 cylinders']

=> fuel <=
['1', '2', '3', '4']

=> title_status <=
['clean', 'rebuilt', 'lien', 'salvage', 'missing', 'parts only']

=> transmission <=
[0, 1, 2]

=> drive <=
[0, 2, 1]

=> type <=
['pickup', 'truck', 'other', 'coupe', 'mini-van', 'SUV', 'sedan', 'hatchback',
'offroad', 'convertible', 'wagon', 'van', 'bus']
```

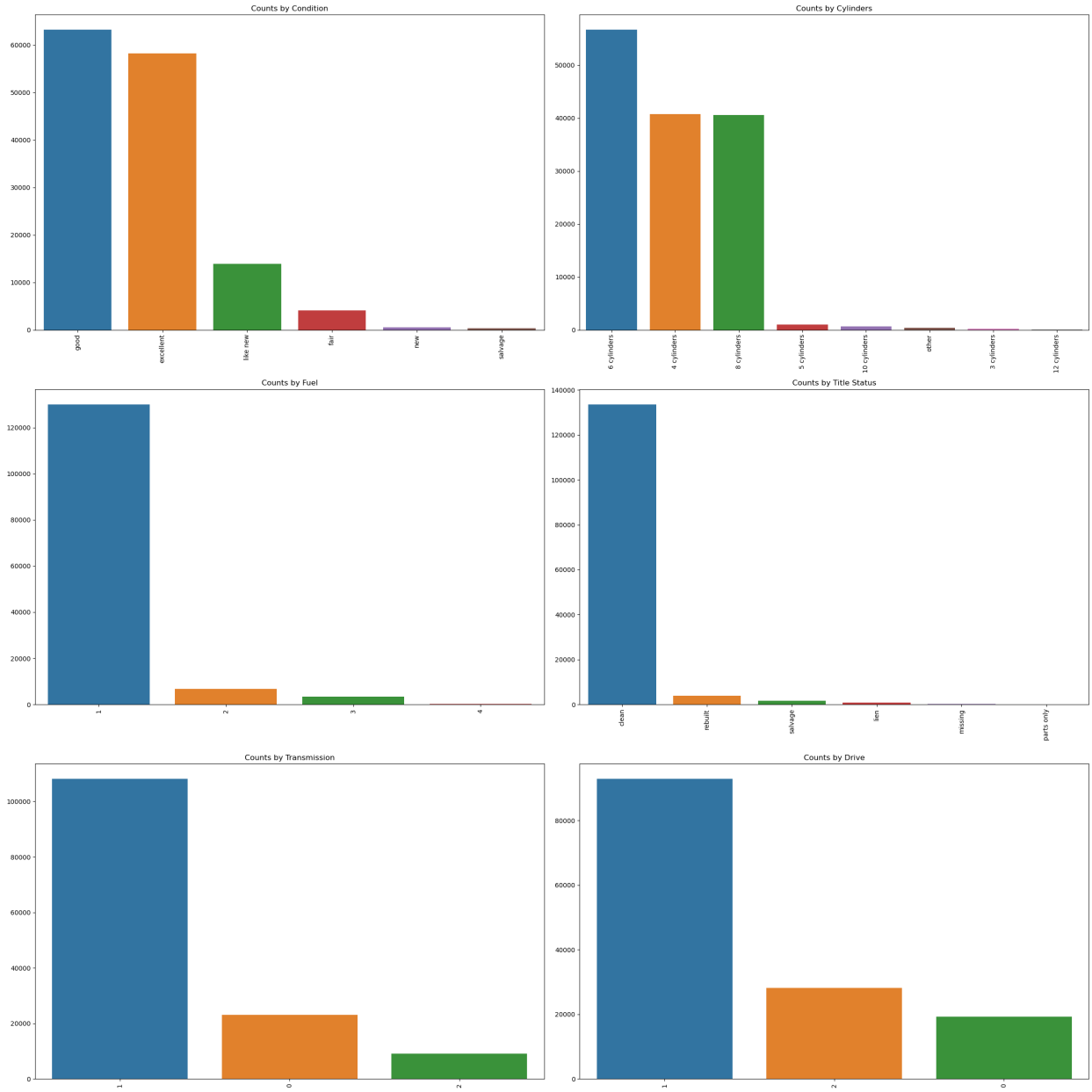
In [ ]:

In [36]:

```
fig, ax = plt.subplots(2, 1, figsize=(24,16))
fig.suptitle('Categorical Features', y=1.)

plot0 = sns.countplot(x='manufacturer', data=df, ax=ax[0], order = df['manufact
plot0.set_title("Counts by Manufacturer")
plot0.set_xticklabels(plot0.get_xticklabels(),rotation = 90)
plot0.set(xlabel=None, ylabel=None)
```

```
plot1 = sns.countplot(x='type', data=df, ax=ax[1],order = df['type'].value_cour
plot1.set_title("Counts by Type")
plot1.set_xticklabels(plot1.get_xticklabels(),rotation = 90)
plot1.set(xlabel=None, ylabel=None)

fig.tight_layout()

# -------------------------
fig, ax = plt.subplots(3, 2, figsize=(24,24))

plot2 = sns.countplot(x='condition', data=df, ax=ax[0,0],order = df['condition'
plot2.set_title("Counts by Condition")
plot2.set_xticklabels(plot2.get_xticklabels(),rotation = 90)
plot2.set(xlabel=None, ylabel=None)

plot3 = sns.countplot(x='cylinders', data=df, ax=ax[0,1],order = df['cylinders'
plot3.set_title("Counts by Cylinders")
plot3.set_xticklabels(plot3.get_xticklabels(),rotation = 90)
plot3.set(xlabel=None, ylabel=None)

plot4 = sns.countplot(x='fuel', data=df, ax=ax[1,0],order = df['fuel'].value_co
plot4.set_title("Counts by Fuel")
plot4.set_xticklabels(plot4.get_xticklabels(),rotation = 90)
plot4.set(xlabel=None, ylabel=None)

plot5 = sns.countplot(x='title_status', data=df, ax=ax[1,1],order = df['title_s
plot5.set_title("Counts by Title Status")
plot5.set_xticklabels(plot5.get_xticklabels(),rotation = 90)
plot5.set(xlabel=None, ylabel=None)
### ------------
### Observation: almost all the titles have the clean status

plot6 = sns.countplot(x='transmission', data=df, ax=ax[2,0],order = df['transmi
plot6.set_title("Counts by Transmission")
plot6.set_xticklabels(plot6.get_xticklabels(),rotation = 90)
plot6.set(xlabel=None, ylabel=None)

plot7 = sns.countplot(x='drive', data=df, ax=ax[2,1],order = df['drive'].value_
plot7.set_title("Counts by Drive")
plot7.set_xticklabels(plot7.get_xticklabels(),rotation = 90)
plot7.set(xlabel=None, ylabel=None)


fig.tight_layout()
```

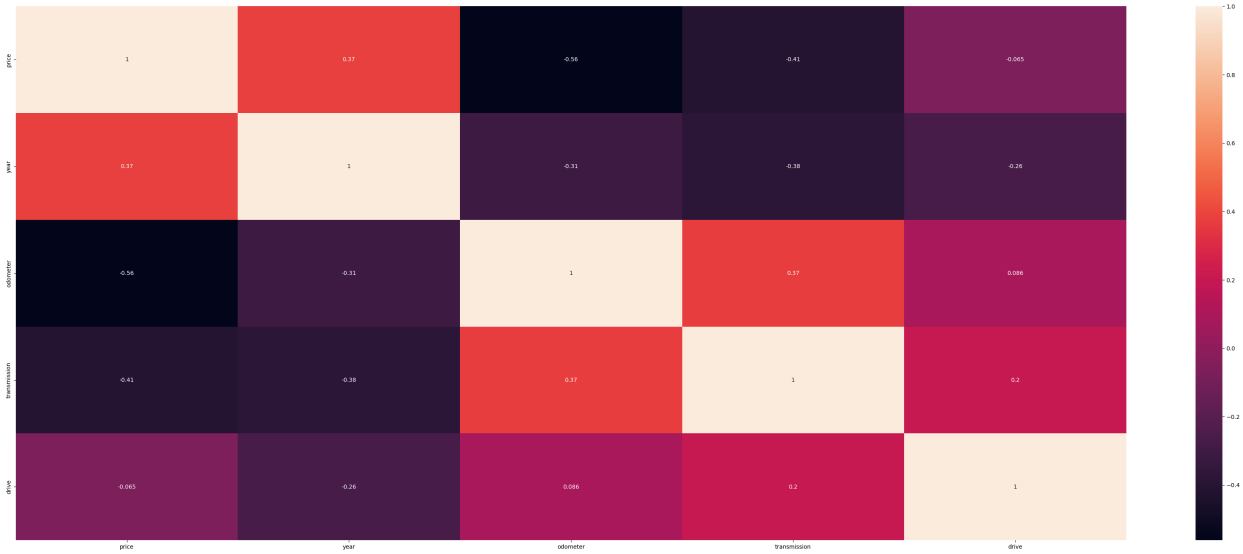

Categorical Features
Counts by Manufacturer
0
5000
10000
15000
20000
ford
chevrolet
toyota
honda
nissan
gmc
jeep
ram
dodge
bmw
mercedes-benz
lexus
hyundai
subaru
volkswagen
chrysler
audi
kia
acura
infiniti
cadillac
buick
lincoln
mazda
pontiac
mitsubishi
volvo
mini
mercury
rover
saturn
jaguar
porsche
fiat
alfa-romeo
tesla
harley-davidson
ferrari
datsun
land rover
aston-martin
Counts by Type
0
5000
10000
15000
20000
25000
30000
35000
sedan
suv
pickup
truck
coupe
other
hatchback
convertible
van
wagon
mini-van
offroad
bus



In [ ]:

In [37]:
```
sns.heatmap(df.corr(), annot=True)
```

Out[37]:
```
<AxesSubplot:>
```



In [ ]:

In [38]:
```
# checking missing value
df.isna().sum()
df.isna().sum().index
```

Out[38]:
```
Index(['price', 'year', 'manufacturer', 'model', 'condition', 'cylinders',
       'fuel', 'odometer', 'title_status', 'transmission', 'type', 'drive'],
      dtype='object')
```

In [39]:
```
# # creagting dummie with one hot encoding
final_df = pd.get_dummies(df,columns = ['price', 'year', 'manufacturer', 'model
       'fuel', 'odometer', 'title_status', 'transmission', 'type', 'drive'], dr
final_df.head()
```

Out[39]:

	price_103	price_114	price_116	price_119	price_120	price_123	price_124	price_127	pric
27	0	0	0	0	0	0	0	0	
28	0	0	0	0	0	0	0	0	
29	0	0	0	0	0	0	0	0	
30	0	0	0	0	0	0	0	0	
31	0	0	0	0	0	0	0	0	

5 rows × 61220 columns

In [ ]:

In [40]:
```
#Converting Categorical Columns to Numerical Columns using Target Encoder. THis
# this is not required after one hot encoding.
# this dataframe will be used in rest of the further process for pipeline creat

from sklearn import preprocessing
from tqdm import tqdm

numeric_col = ['price', 'year',  'transmission', 'drive', 'odometer']
categorical_col = ['manufacturer', 'model','cylinders', 'condition', 'fuel', 't
```

```
#Converting Categorical Columns to Numerical Columns using Target Encoder
encoder_df=df.copy()

encoder=preprocessing.LabelEncoder()

def encode(data):
    noNaN = np.array(data.dropna())
    impute_ordinal = encoder.fit_transform(noNaN.reshape(-1,1))
    data.loc[data.notnull()] = np.squeeze(impute_ordinal)
    return data

for i in tqdm(range(len(categorical_col))):
    encode(encoder_df[categorical_col[i]])
```

```
100%|██████████████████████████████████████| 7/7 [00:00<00:00, 19.73it/
s]
```

In [41]:
```
encoder_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 140442 entries, 27 to 426878
Data columns (total 12 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   price         140442 non-null  int64
 1   year          140442 non-null  float64
 2   manufacturer  140442 non-null  object
 3   model         140442 non-null  object
 4   condition     140442 non-null  object
 5   cylinders     140442 non-null  object
 6   fuel          140442 non-null  object
 7   odometer      140442 non-null  float64
 8   title_status  140442 non-null  object
 9   transmission  140442 non-null  int64
 10  type          140442 non-null  object
 11  drive         140442 non-null  int64
dtypes: float64(2), int64(3), object(7)
memory usage: 13.9+ MB
```

In [42]:
```
encoder_df.head()
```

Out[42]:

	price	year	manufacturer	model	condition	cylinders	fuel	odometer	title_status	tra
27	33590	2014.0	14	9921	2	6	0	57923.0	0	
28	22590	2010.0	7	10095	2	6	0	71229.0	0	
29	39590	2020.0	7	10108	2	6	0	19160.0	0	
30	30990	2017.0	38	11632	2	6	0	41124.0	0	
31	15000	2013.0	13	5089	0	5	0	128000.0	0	

Modeling

With your (almost?) final dataset in hand, it is now time to build some models. Here, you should build a number of different regression models with the price as the target. In building

your models, you should explore different parameters and be sure to cross-validate your findings.

In [43]:
```
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import RidgeCV
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures, OneHotEncoder, OrdinalEnc
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split, GridSearchCV, LeaveOneOut
from sklearn.compose import make_column_transformer, make_column_selector
from sklearn import set_config

set_config(display="diagram") #setting this will display the pipelines
```

In [44]:
```
X = encoder_df.drop(columns=['price']) #input features excluding target variabl
y = encoder_df['price']  #target variable

# split into test and train data set
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state = 42)

print(f'{X_train.shape},{X_test.shape}')
print(f'{y_train.shape},{y_test.shape}')
print(type(X_train))
print(type(y_train))
```

```
(105331, 11),(35111, 11)
(105331,),(35111,)
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
```

In [45]:
```
# %%time
# # ******* Linear Regression ********

# # import the required library
# from sklearn.linear_model import LinearRegression
# from sklearn.metrics import mean_squared_error
# from sklearn.metrics import r2_score

# # initiate and fit
# linreg = LinearRegression()
# linreg.fit(X_train, y_train)
# y_pred = linreg.predict(X_test)
# mse = mean_squared_error(y_test, y_pred)
# rmse = np.sqrt(mse)
# r2 = r2_score(y_test, y_pred)

# print(f'LinearReg MSE: {mse}')
# print(f'LinearReg RMSE: {rmse}')
# print(f'LinearReg R2 Score: {r2}')
```

In [46]:
```
%%time
# ******* Linear Regression ********
```

```
# initiate and fit

linear_pipe = Pipeline([('scale', StandardScaler()),
                        ('linreg', LinearRegression())])

linear_pipe.fit(X_train, y_train)
ln_train_preds = linear_pipe.predict(X_train)
ln_y_preds = linear_pipe.predict(X_test)
# ln_train_mse = mean_squared_error(y_train, ln_y_preds)
ln_test_mse = mean_squared_error(y_test, ln_y_preds)
ln_train_r2score = r2_score(y_train, ln_train_preds)
ln_test_r2score = r2_score(y_test, ln_y_preds)

#print train and test scores for the model
print ('\nLinear Regression Model: Model Scores and Coefficients\n\n')
# print('Train MSE:',np.around(ln_train_mse,2))
print('Train r2 score:', np.around(ln_train_r2score,2))
print('*==============*')
print('Test MSE:',np.around(ln_test_mse,2))
print('Test RMSE:', np.around(np.sqrt(ln_test_mse),2))
print('Test r2 score:', np.around(ln_test_r2score,2))
```

```
Linear Regression Model: Model Scores and Coefficients


Train r2 score: 0.51
*==============*
Test MSE: 92624340.72
Test RMSE: 9624.15
Test r2 score: 0.51
CPU times: user 714 ms, sys: 43.8 ms, total: 757 ms
Wall time: 346 ms
```

In [64]:

```
# coefficients of the model
ln_coefs = linear_pipe.named_steps['linreg'].coef_

list_ln_coeff = list((zip(linear_pipe.named_steps['scale'].get_feature_names_ou

df_ln_coeff = pd.DataFrame(list_ln_coeff, columns = [' Features', 'linear Coeff

#sort features by magnitude of linear regression coefficents
df_ln_coeff.sort_values(by='linear Coefficients', ascending=False, key=abs)
```

Out[64]:

	Features	linear Coefficients
6	odometer	-6208.387329
4	cylinders	4273.269345
0	year	3330.992993
5	fuel	1910.994485
8	transmission	-821.819268
7	title_status	-819.017386
3	condition	379.485919
1	manufacturer	325.943660
2	model	302.227352
10	drive	224.729391
9	type	96.254836

In [ ]:

In [48]:

```
# Ridge Regression with GridSearchCV for the 'best' alpha

# building a Ridge Regression model using GridSearchCV for tuning hyperparamete

pipeRidge = Pipeline( [('scale', StandardScaler()),('ridge', Ridge())])
param_dict = {'ridge__alpha': [0.001, 0.1, 1.0, 10.0, 100.0, 500.0, 1000.0]}
gridRidge = GridSearchCV(pipeRidge, param_grid=param_dict)
gridRidge.fit(X_train, y_train)
train_predsRidge = gridRidge.predict(X_train)
test_predsRidge = gridRidge.predict(X_test)
# train_mseRidge = mean_squared_error(y_train, train_predsRidge)
train_r2scoreRidge = r2_score(y_train, train_predsRidge)
test_mseRidge = mean_squared_error(y_test, test_predsRidge)
test_r2scoreRidge = r2_score(y_test, test_predsRidge)
best_alphaRidge = gridRidge.best_params_

# print best alpha and model train and test scores
print ('\nRidge Regression Model: Best Alpha and Model Scores\n\n')
print(f'Best Alpha: {list(best_alphaRidge.values())[0]}')
# print('Train MSE:',np.around(train_mseRidge,2))
print('Train r2 score:', np.around(train_r2scoreRidge,2))
print('Test MSE:',np.around(test_mseRidge,2))
print('Test RMSE:', np.around(np.sqrt(test_mseRidge),2))
print('Test r2 score:', np.around(test_r2scoreRidge,2))
```

```
Ridge Regression Model: Best Alpha and Model Scores


Best Alpha: 10.0
Train r2 score: 0.51
Test MSE: 92624166.29
Test RMSE: 9624.14
Test r2 score: 0.51
```

In [49]:

```
#Inspect the coefficients of the ridge model with best alpha=10.0
pipe_bestalpha_ridge = Pipeline([('scale', StandardScaler()), ('ridge', Ridge(a
```

```
pipe_bestalpha_ridge.fit(X_train, y_train)


list_Ridge_coeff = list((zip(pipe_bestalpha_ridge.named_steps['scale'].get_feat

df_Ridge_coeff = pd.DataFrame(list_Ridge_coeff, columns = [' Features', 'Ridge

print('\nRidge Regression Coefficients\n\n')

#sort features by magnitude of ridge regression coefficents
df_Ridge_coeff.sort_values(by='Ridge Coefficients', ascending=False, key=abs)
```

```
Ridge Regression Coefficients
```

Out[49]:

	Features	Ridge Coefficients
6	odometer	-6207.752925
4	cylinders	4272.735863
0	year	3330.667291
5	fuel	1910.886367
8	transmission	-822.222315
7	title_status	-818.927779
3	condition	379.520740
1	manufacturer	325.779300
2	model	302.249087
10	drive	224.711711
9	type	96.273103

In [ ]:

In [50]:

```
# Lasso Model with GridSearchCV for optimal lasso alpha
#Next implementing the lasso model : picking the best alpha parameter for the l

lasso_alphas = [0.001,0.01,0.1,1.0, 10.0]
lasso = Lasso(random_state=123)
grid = dict()
grid['alpha'] = lasso_alphas

gscv_lasso = GridSearchCV( \
    lasso, grid, scoring='neg_mean_squared_error', \
    cv=3, n_jobs=-1)

gscv_lasso.fit(X_train, y_train)
train_predsLasso = gscv_lasso.predict(X_train)
test_predsLasso = gscv_lasso.predict(X_test)
train_mseLasso = mean_squared_error(y_train, train_predsLasso)
train_r2scoreLasso = r2_score(y_train, train_predsLasso)
test_mseLasso = mean_squared_error(y_test, test_predsLasso)
test_r2scoreLasso = r2_score(y_test, test_predsLasso)
best_alphaLasso = gscv_lasso.best_params_
```

```
# print best alpha and model train and test scores for Lasso model
print ('\nLasso Model: Best Alpha and Model Scores\n\n')
print(f'Best Alpha: {list(best_alphaLasso.values())[0]}')
print('Train MSE:',np.around(train_mseLasso,2))
print('Train r2 score:', np.around(train_r2scoreLasso,2))
print('Test MSE:',np.around(test_mseLasso,2))
print('Test RMSE:', np.around(np.sqrt(test_mseLasso),2))
print('Test r2 score:', np.around(test_r2scoreLasso,2))
```

```
Lasso Model: Best Alpha and Model Scores


Best Alpha: 0.001
Train MSE: 93655698.11
Train r2 score: 0.51
Test MSE: 92624339.54
Test RMSE: 9624.15
Test r2 score: 0.51
```

In [51]:

```
#Inspect the coefficients of the Lasso model with best alpha=0.001
pipe_bestalpha_lasso = Pipeline([('scale', StandardScaler()), ('lasso', Lasso(a
pipe_bestalpha_lasso.fit(X_train, y_train)

list_Lasso_coeff = list((zip(pipe_bestalpha_lasso.named_steps['scale'].get_feat

df_Lasso_coeff = pd.DataFrame(list_Lasso_coeff, columns = [' Features', 'Lasso

print('\nLasso Coefficients\n\n')

#sort features by magnitude of lasso coefficents
df_Lasso_coeff.sort_values(by='Lasso Coefficients', ascending=False, key=abs)
```

```
Lasso Coefficients
```

Out[51]:

	Features	Lasso Coefficients
6	odometer	-6208.386611
4	cylinders	4273.268601
0	year	3330.992082
5	fuel	1910.993823
8	transmission	-821.819058
7	title_status	-819.016428
3	condition	379.485229
1	manufacturer	325.942259
2	model	302.226224
10	drive	224.727920
9	type	96.253959

In [52]:

```
# Sequential Feature Selection using Linear Regression Estimator
from sklearn.feature_selection import SequentialFeatureSelector, SelectFromMode

#SequentialFeature selector with number of features = 5 and LinearRegression es
```

```
pipeSFSelector = Pipeline([('scale', StandardScaler()),('linreg', LinearRegress
SFSelector = SequentialFeatureSelector(pipeSFSelector, n_features_to_select=5)

#best four features from Sequential Feature Selection
best_features_SFSelector = SFSelector.fit_transform(X_train, y_train)
best_featuresSFSelector_df = pd.DataFrame(best_features_SFSelector, columns = S
best_featuresSFSelector_df.head()
```

Out[52]:

	year	cylinders	fuel	odometer	title_status
0	2014.0	6	0	53263.0	0
1	2010.0	3	0	133254.0	0
2	2009.0	5	0	156022.0	0
3	1982.0	6	0	140000.0	0
4	2007.0	5	0	225752.0	0

In [53]:

```
#pipeline with column selector from Sequential Feature Selection
pipelineSFSLR = Pipeline([('column_selector', SFSelector),
                  ('scale', StandardScaler()),
                ('linreg', LinearRegression())])

pipelineSFSLR.fit(X_train, y_train)
train_preds_SFSLR = pipelineSFSLR.predict(X_train)
y_preds_SFSLR = pipelineSFSLR.predict(X_test)
train_mse_SFSLR = mean_squared_error(y_train, train_preds_SFSLR)
train_r2score_SFSLR = r2_score(y_train, train_preds_SFSLR)
test_mse_SFSLR = mean_squared_error(y_test, y_preds_SFSLR)
test_r2score_SFSLR = r2_score(y_test, y_preds_SFSLR)

# print  model train and test scores for SFS with LR
print ('\nSequential Feature Selection With Linear Regression Estimator: Model
print('Train MSE:',np.around(train_mse_SFSLR,2))
print('Train r2 score:', np.around(train_r2score_SFSLR,2))
print('Test MSE:',np.around(test_mse_SFSLR,2))
print('Test RMSE:', np.around(np.sqrt(test_mse_SFSLR),2))
print('Test r2 score:', np.around(test_r2score_SFSLR,2))


list_SFSLR_coeff = list((zip(pipelineSFSLR['column_selector'].get_feature_names

df_SFSLR_coeff = pd.DataFrame(list_SFSLR_coeff, columns = ['SFS Features', 'Moc

#sort features by magnitude of model coefficents
df_SFSLR_coeff.sort_values(by='Model Coefficients', ascending=False, key=abs)
```

```
Sequential Feature Selection With Linear Regression Estimator: Model Scores an
d Coefficients


Train MSE: 94535950.13
Train r2 score: 0.51
Test MSE: 93546280.41
Test RMSE: 9671.93
Test r2 score: 0.5
```

Out[53]:

	SFS Features	Model Coefficients
3	odometer	-6438.948779
1	cylinders	4500.433135
0	year	3582.412372
2	fuel	1965.733629
4	title_status	-871.951460

In [54]:

```
# Sequential Feature Selection showed how a simpler model can built with fewer

#     Model (positive)

#     Odomoter (negative)

#     Year (positive)

#     Transmission (negative)

#     Fuel (positive)
```

In [55]:

```
# Higher Order Regression Models with Polynomial Features

# Next we evaluate the effect of increasing complexity with PolynomialFeatures
# complexites from degree = 1 through degree = 4 and a LinearRegession estimato
# based on the test mean squared error.
```

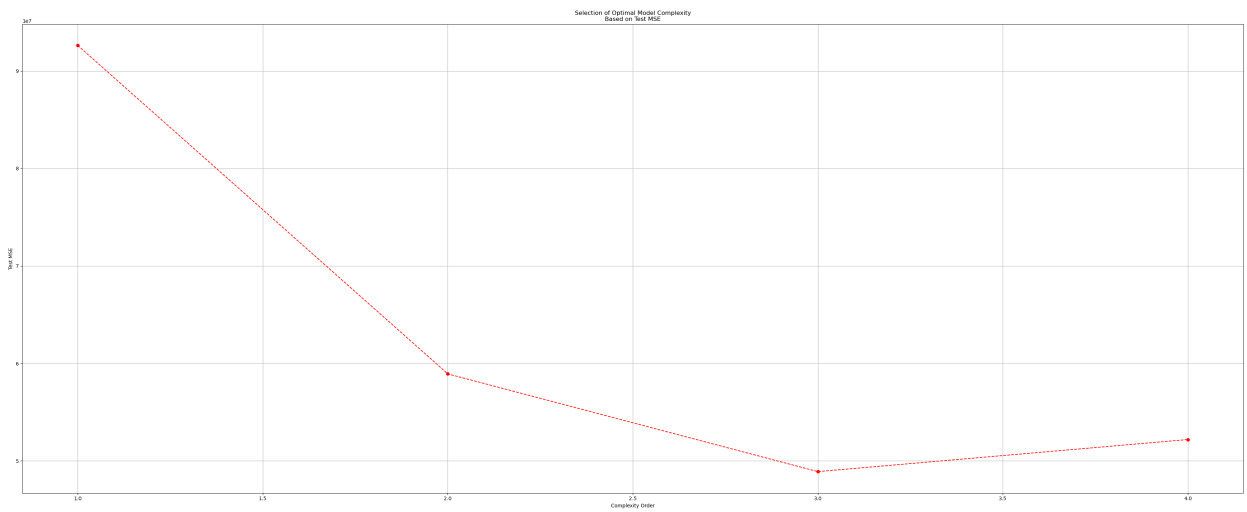
In [56]:

```
# building a pipeline model with polynomial features and linear regression to t
# complexity

train_mses = []
train_r2score=[]
test_mses = []
test_r2score=[]
for i in range(1, 5):
    pipe = Pipeline([('pfeat', PolynomialFeatures(degree = i, include_bias=Fals
                     ('scale', StandardScaler()),
                     ('linreg', LinearRegression())])

    pipe.fit(X_train, y_train)
    train_preds = pipe.predict(X_train)
    y_preds = pipe.predict(X_test)
    train_mses.append(mean_squared_error(y_train, train_preds))
    test_mses.append(mean_squared_error(y_test, y_preds))
    train_r2score.append(r2_score(y_train, train_preds))
    test_r2score.append(r2_score(y_test, y_preds))


# find the best model complexity best on the testing error (and not training er
best_model_complexity = test_mses.index(min(test_mses)) + 1

print('\nLinear Regression with Higher Order Polynomials\n')
print('\n\nBest complexity:' , best_model_complexity)
print('\nTest MSEs:\n', np.around(test_mses,2))
print('\nTest RMSEs:\n', np.around(np.sqrt(test_mses),2))
print('\nTest r2 scores:\n', np.around(test_r2score,2))
```

```
plt.plot(np.arange(1,5), test_mses, 'ro--')
plt.grid()
plt.xlabel('Complexity Order')
plt.ylabel('Test MSE')
plt.title('Selection of Optimal Model Complexity\nBased on Test MSE')
```

```
Linear Regression with Higher Order Polynomials



Best complexity: 3

Test MSEs:
 [92624340.72 58922805.81 48899768.1  52191560.01]

Test RMSEs:
 [9624.15 7676.12 6992.84 7224.37]

Test r2 scores:
 [0.51 0.69 0.74 0.72]
```

Out[56]:
```
Text(0.5, 1.0, 'Selection of Optimal Model Complexity\nBased on Test MSE')
```



In [65]:
```
linreg_coeffs = pd.Series(linear_pipe.named_steps['linreg'].coef_, index=X_trai
linreg_coeffs.sort_values(ascending=False)
```

Out[65]:
```
cylinders       4273.269345
year            3330.992993
fuel            1910.994485
condition        379.485919
manufacturer     325.943660
model            302.227352
drive            224.729391
type              96.254836
title_status    -819.017386
transmission    -821.819268
odometer       -6208.387329
dtype: float64
```

In [66]:
```
# Ridge Regression with Higher Order Polynomial (degree =3) and GridSearchCV fo

pipe_Ridge = Pipeline( [('pfeat', PolynomialFeatures(degree = 3, include_bias=F
param_dict = {'ridge__alpha': [0.001, 0.1, 1.0, 10.0, 100.0]}
grid_Ridge = GridSearchCV(pipe_Ridge, param_grid=param_dict)
```

```
grid_Ridge.fit(X_train, y_train)
train_preds_Ridge = grid_Ridge.predict(X_train)
test_preds_Ridge = grid_Ridge.predict(X_test)
train_mse_Ridge = mean_squared_error(y_train, train_preds_Ridge)
train_r2score_Ridge = r2_score(y_train, train_preds_Ridge)
test_mse_Ridge = mean_squared_error(y_test, test_preds_Ridge)
test_r2score_Ridge = r2_score(y_test, test_preds_Ridge)
Ridge_best_alpha = grid_Ridge.best_params_

# print best alpha and corresponding train/test scores
print('\n Ridge Regression with Higher Order Polynomial (degree=3) : Best Alpha
print(f'Best Alpha: {list(Ridge_best_alpha.values())[0]}')
print(f'Train MSE: {np.around(train_mse_Ridge,2)}')
print(f'Train r2 score: {np.around(train_r2score_Ridge,2)}')
print(f'Test MSE: {np.around(test_mse_Ridge,2)}')
print(f'Test RMSE: {np.around(np.sqrt(test_mse_Ridge),2)}')
print(f'Test r2 score: {np.around(test_r2score_Ridge,2)}')
```

```
 Ridge Regression with Higher Order Polynomial (degree=3) : Best Alpha and Mod
el Scores

Best Alpha: 0.001
Train MSE: 50457283.94
Train r2 score: 0.74
Test MSE: 51626053.04
Test RMSE: 7185.13
Test r2 score: 0.73
```

In [67]:

```
ridgereg = Ridge(alpha=0.001).fit(X_train, y_train)
ridgereg_coeffs = pd.Series(ridgereg.coef_, index=X_train.columns)
ridgereg_coeffs.sort_values(ascending=False)
```

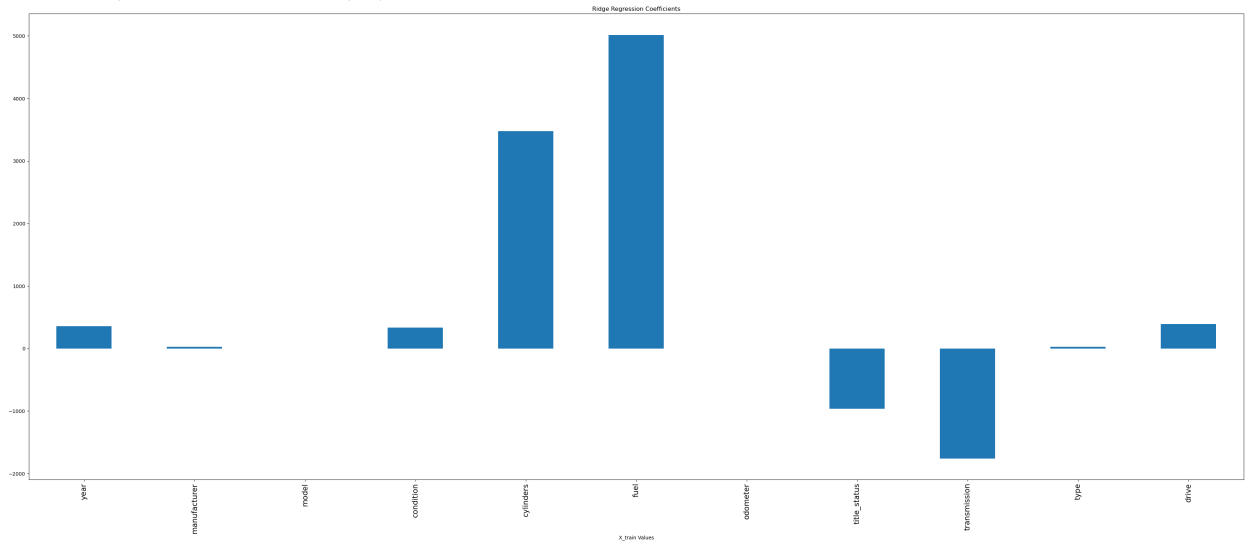
Out[67]:

```
fuel            5015.709668
cylinders       3475.902266
drive            389.245108
year             356.806038
condition        336.200483
manufacturer      29.048631
type              24.205757
model              0.084927
odometer          -0.090922
title_status    -965.134278
transmission   -1758.543573
dtype: float64
```

In [68]:

```
ridgereg_coeffs.plot(kind='bar')
plt.title('Ridge Regression Coefficients')
plt.xlabel('X_train Values')
plt.xticks(size=15)
```

Out[68]:
```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10]),
 [Text(0, 0, 'year'),
  Text(1, 0, 'manufacturer'),
  Text(2, 0, 'model'),
  Text(3, 0, 'condition'),
  Text(4, 0, 'cylinders'),
  Text(5, 0, 'fuel'),
  Text(6, 0, 'odometer'),
  Text(7, 0, 'title_status'),
  Text(8, 0, 'transmission'),
  Text(9, 0, 'type'),
  Text(10, 0, 'drive')])
```



In [69]:
```
# Lasso with higher order polynomial and GridSearchCV for optimal alpha

pipelasso = Pipeline([('pfeat', PolynomialFeatures(degree = 3, include_bias=Fal
                      ('lasso', Lasso(random_state=123))])
param_dict = {'lasso__alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]}



gscv_pipelasso = GridSearchCV( \
    pipelasso, param_grid=param_dict, scoring='neg_mean_squared_error', \
    cv=3, n_jobs=-1)

gscv_pipelasso.fit(X_train, y_train)
train_preds_pipelasso = gscv_pipelasso.predict(X_train)
test_preds_pipelasso = gscv_pipelasso.predict(X_test)
train_mse_pipelasso = mean_squared_error(y_train, train_preds_pipelasso)
train_r2score_pipelasso = r2_score(y_train, train_preds_pipelasso)
test_mse_pipelasso = mean_squared_error(y_test, test_preds_pipelasso)
test_r2score_pipelasso = r2_score(y_test, test_preds_pipelasso)
best_alpha_pipelasso = gscv_pipelasso.best_params_

# print best alpha and model train and test scores for Lasso model
print ('\nLasso Model with 3nd order Polynomial Features: Best Alpha and Model
print(f'Best Alpha: {list(best_alpha_pipelasso.values())[0]}')
print('Train MSE:',np.around(train_mse_pipelasso,2))
print('Train r2 score:', np.around(train_r2score_pipelasso,2))
print('Test MSE:',np.around(test_mse_pipelasso,2))
print('Test RMSE:', np.around(np.sqrt(test_mse_pipelasso),2))
print('Test r2 score:', np.around(test_r2score_pipelasso,2))
```

```
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.198e+12, tolerance:
1.353e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.148e+12, tolerance:
1.335e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.196e+12, tolerance:
1.353e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.147e+12, tolerance:
1.335e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.181e+12, tolerance:
1.344e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.193e+12, tolerance:
1.344e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.192e+12, tolerance:
1.344e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.186e+12, tolerance:
1.353e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 3.040e+09, tolerance:
1.344e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 6.310e+11, tolerance:
1.344e+09
  model = cd_fast.enet_coordinate_descent(
```

```
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 5.541e+11, tolerance:
1.335e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 6.327e+11, tolerance:
1.353e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.064e+12, tolerance:
1.344e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.075e+12, tolerance:
1.353e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.017e+12, tolerance:
1.335e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.136e+12, tolerance:
1.335e+09
  model = cd_fast.enet_coordinate_descent(
/Users/mohul1/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_
coordinate_descent.py:647: ConvergenceWarning: Objective did not converge. You
might want to increase the number of iterations, check the scale of the featur
es or consider increasing regularisation. Duality gap: 2.475e+09, tolerance:
1.335e+09
  model = cd_fast.enet_coordinate_descent(
```

```
Lasso Model with 3nd order Polynomial Features: Best Alpha and Model Scores


Best Alpha: 1.0
Train MSE: 62301385.72
Train r2 score: 0.67
Test MSE: 63701043.83
Test RMSE: 7981.29
Test r2 score: 0.66
```

In [ ]:

```

```

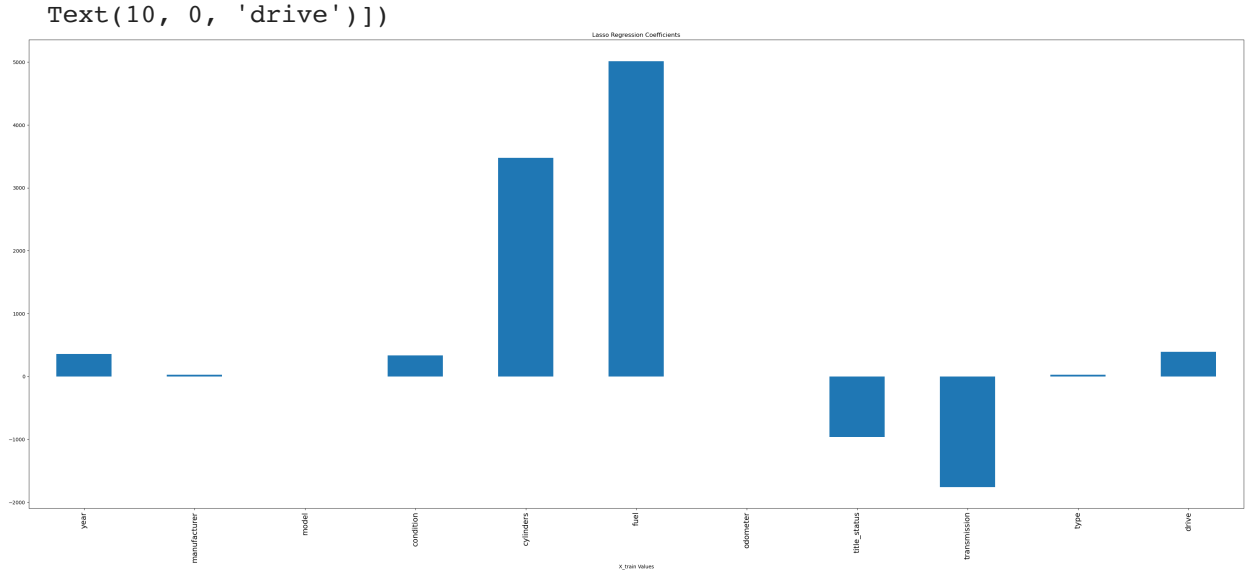
In [70]:

```
lassoreg = Lasso(alpha=0.001).fit(X_train, y_train)
lassoreg_coeffs = pd.Series(lassoreg.coef_, index=X_train.columns)
lassoreg_coeffs.sort_values(ascending=False)
```

Out[70]:
```
fuel            5015.703382
cylinders       3475.902776
drive            389.241025
year             356.806069
condition        336.200116
manufacturer      29.048602
type              24.205807
model              0.084926
odometer          -0.090922
title_status    -965.133182
transmission   -1758.537778
dtype: float64
```

In [71]:
```
lassoreg_coeffs.plot(kind='bar')
plt.title('Lasso Regression Coefficients')
plt.xlabel('X_train Values')
plt.xticks(size=15)
```

Out[71]:
```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10]),
 [Text(0, 0, 'year'),
  Text(1, 0, 'manufacturer'),
  Text(2, 0, 'model'),
  Text(3, 0, 'condition'),
  Text(4, 0, 'cylinders'),
  Text(5, 0, 'fuel'),
  Text(6, 0, 'odometer'),
  Text(7, 0, 'title_status'),
  Text(8, 0, 'transmission'),
  Text(9, 0, 'type'),
  Text(10, 0, 'drive')])
```



In [ ]:

Evaluation

With some modeling accomplished, we aim to reflect on what we identify as a high quality model and what we are able to learn from this. We should review our business objective and explore how well we can provide meaningful insight on drivers of used car prices. Your goal now is to distill your findings and determine whether the earlier phases need revisitation and adjustment or if you have information of value to bring back to your client.

In [72]:
```
#Identifying MSE to choose n_neighbors
from sklearn.neighbors import KNeighborsRegressor

mses=[]
for i in range(1,10):
    KNN=KNeighborsRegressor(n_neighbors=i)
    KNN.fit(X_train,y_train)
    y_pred=KNN.predict(X_test)
    results=np.sqrt(mean_squared_error(y_test, y_pred))
    mses.append(results)
print(mses)
```

```
[9154.53068571684, 8582.55516928429, 8373.722288434754, 8389.954364515888, 843
8.479356185637, 8511.306141823366, 8583.560680950055, 8639.658433590534, 8692.
053636612112]
```

In [ ]:

In [73]:
```
KNN=KNeighborsRegressor(n_neighbors=2)
KNN.fit(X_train,y_train)
y_pred3=KNN.predict(X_test)


mseKNN = mean_squared_error(y_test, y_pred3)
rmseKNN = np.sqrt(mseKNN)
r2_KNN = r2_score(y_test, y_pred3)

# KNN_best_alpha = lasso_grid.best_params_

print(f'KNN MSE: {mseKNN}')
print(f'KNN RMSE: {rmseKNN}')
print(f'KNN R2 Score: {r2_KNN}')
# print(f'LassoReg Best Alpha: {lasso_best_alpha}')



# KNN_results = model_scores(y_test, y_pred3)
# print(f'KNN MSE: {KNN_results[0]}')
# print(f'KNN RMSE: {KNN_results[1]}')
# print(f'KNN R2 Score: {KNN_results[2]}')

# model_results.append({'Model_name': 'KNN Regression', 'MSE': KNN_results[0],
```

```
KNN MSE: 73660253.23380849
KNN RMSE: 8582.55516928429
KNN R2 Score: 0.6082640696407685
```

In [74]:
```
# Our best model is second order polynomial model with RIDGE Regression that ac
# predict the price of used cars. We have used hold out cross-validation for ou
# In the evaluation phase, we want to re-test the performance of the model usir
# results from the hold out cross-validation are not biased due to any unpreced
# sure test datasets are encoded without utilization of the test values of the
# unbiased results from K-Fold Cross-Validation
```

In [77]:
```
pip install category_encoders
# installed category ecoder for K-fold validation
```

In [76]:
```
import category_encoders as ce
```

In [78]:
```
# Define the number of folds
n_splits = 5

# Define the cross-validation scheme
kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)

test_mse_KF =[]
test_rmse_KF=[]
test_r2score_KF=[]

# Define the encoder
encoder = ce.LeaveOneOutEncoder(return_df=True)



# Loop over the folds
for train_index, test_index in kf.split(X):
    # Get the training and test data for this fold
    X_trainKF, X_testKF = X.iloc[train_index], X.iloc[test_index]
    y_trainKF, y_testKF = y.iloc[train_index], y.iloc[test_index]

    # Fit the encoder on the training data for this fold only
    encoder.fit(X_trainKF, y_trainKF)

    # Encode the categorical variables in the training and test data
    X_train_encodedKF = encoder.transform(X_trainKF)
    X_test_encodedKF = encoder.transform(X_testKF)

    # rigde is the best model so
    pipe_BestModel = pipe_Ridge

    # Fit the best model on the encoded training data
    pipe_BestModel.fit(X_train_encodedKF, y_trainKF)
    # Make predictions on the encoded test data
    y_predKF = pipe_BestModel.predict(X_test_encodedKF)

    # Calculate the test scores for this fold
    test_mse_KF.append(np.around(mean_squared_error(y_testKF, y_predKF),2))
    test_rmse_KF.append(np.around(np.sqrt(mean_squared_error(y_testKF, y_predKF
    test_r2score_KF.append(np.around(r2_score(y_testKF, y_predKF),2))


print('\nKFold Scores for number of splits = 5\n')
print('Test MSEs: \n', test_mse_KF)
print('Mean Test MSE:', np.around(np.mean(test_mse_KF),2))
print('\nTest RMSEs: \n', test_rmse_KF)
print('Mean Test RMSE:', np.around(np.mean(test_rmse_KF),2))
print('\nTest r2 scores:\n', test_r2score_KF)
print('Mean Test r2 score:', np.around(np.mean(test_r2score_KF),2))
```

```
KFold Scores for number of splits = 5

Test MSEs:
 [40693089.59, 40896091.31, 37258684.03, 40379998.26, 42239536.8]
Mean Test MSE: 40293480.0

Test RMSEs:
 [6379.11, 6395.01, 6103.99, 6354.53, 6499.2]
Mean Test RMSE: 6346.37

Test r2 scores:
 [0.78, 0.79, 0.8, 0.79, 0.78]
Mean Test r2 score: 0.79
```

In [79]:

```
# The result from the K-FOLD is similar to the output from the earlier hold out
# This shows that the model is consistent with test & train data set
```

In [80]:

```
from sklearn.inspection import permutation_importance

scoring = ['r2', 'neg_mean_absolute_percentage_error', 'neg_mean_squared_error'
r_multi = permutation_importance(pipe_BestModel, X_test, y_test, n_repeats=30,

for metric in r_multi:
    print(f"{metric}")
    r = r_multi[metric]
    for i in r.importances_mean.argsort()[::-1]:
            if r.importances_mean[i] - 2 * r.importances_std[i] > 0:
               print(f"    {X_test.columns[i]:<8}:   "
                       f"{r.importances_mean[i]:.3f}"
                           f" +/- {r.importances_std[i]:.3f}")
```

```
r2
    odometer:   0.192 +/- 0.010
    transmission:   0.037 +/- 0.009
    type    :   0.000 +/- 0.000
    cylinders:   0.000 +/- 0.000
    condition:   0.000 +/- 0.000
neg_mean_absolute_percentage_error
    odometer:   0.549 +/- 0.018
    transmission:   0.510 +/- 0.022
    drive   :   0.069 +/- 0.009
    year    :   0.020 +/- 0.006
    manufacturer:   0.000 +/- 0.000
    condition:   0.000 +/- 0.000
neg_mean_squared_error
    odometer:   36122395.876 +/- 1915043.831
    transmission:   6867157.473 +/- 1730809.322
    type    :   33322.472 +/- 3887.622
    cylinders:   12907.541 +/- 946.076
    condition:   11677.192 +/- 306.833
```

Recap:

The start was with pulling out used car data with a goal to identify the key factor which influence the price. Along with that, we need to build a model for dealrship to adjust their used car inventory based on the features which are highly desired. Next, data was reveiwed,

identified outliers, cleaned them up and replace catgorical features with numerical features nd applied encoder. After that, used the train and test set of data on various models and complexities to observe the higher accuracy.

In this analysis, RIDGE model with higher Order Polynomial with complexcity degree=3 resulted in higher accuracy as 73%.

Deployment

Now that we've settled on our models and findings, it is time to deliver the information to the client. You should organize your work as a basic report that details your primary findings. Keep in mind that your audience is a group of used car dealers interested in fine tuning their inventory.

In [ ]:

```

```

Introduction:

This report aims to provide insights into the top 3 key drivers affecting the pricing of used cars. Our objective is to help used car dealers fine-tune their inventory and pricing strategies based on the findings of our analysis.

Methodology:

We analyzed a dataset of used car transactions, focusing on the different attributes such as Model, Manufacturer, Year, Odometer, Title Status, Fuel, Cylinder, and Drive. Our analysis involved performing a multiple regression analysis, where we modeled the sale price of a used car as a function of these attributes. We also used data visualization techniques to explore the relationship between these attributes and the sale price.

Findings:

Our analysis revealed the following key findings:

Fuel type: The type of fuel used by a car is a significant driver of its price. Cars that run on diesel or hybrid fuel tend to have higher sale prices compared to cars that run on gasoline.

Cylinder count: The number of cylinders in a car's engine is another key driver of its price. Cars with more cylinders tend to have higher sale prices compared to cars with fewer cylinders.

Drive type: The drive type of a car also plays a significant role in determining its price. Cars with all-wheel drive (AWD) or four-wheel drive (4WD) tend to have higher sale prices compared to cars with front-wheel drive (FWD) or rear-wheel drive (RWD).

Recommendations:

Based on our findings, we recommend that used car dealers consider the Fuel, Cylinder, and Drive attributes when setting prices for their inventory. Cars that run on diesel or hybrid fuel, have more cylinders, or feature AWD or 4WD are likely to command higher prices in the market. Dealers should also consider adjusting their pricing strategies for cars with these attributes to better reflect their market value.

Conclusion:

In conclusion, our analysis highlights the importance of considering key attributes such as Fuel, Cylinder, and Drive when determining the pricing of used cars. By taking these attributes into account, used car dealers can fine-tune their inventory and pricing strategies to better align with market demand, ultimately leading to more successful sales and greater profitability.

In [ ]:

```

```

In [ ]:

```

```